

Guia de instalacion y preparacion del entorno CMSIS

TP2 – Ejercicio 3

Tecnicas Digitales II – UTN FRC

1. Objetivo

El objetivo de esta guía es facilitar la instalación, organización y configuración de las librerías necesarias para migrar el código desarrollado en el TP2 hacia una implementación basada en **CMSIS** sobre la placa **BluePill STM32F103C8T6**.

A lo largo del documento se describirá cómo descargar, organizar e integrar los archivos necesarios de **CMSIS** para poder migrar el código desarrollado previamente usando punteros manuales hacia una implementación basada en las estructuras definidas por CMSIS. Por ejemplo, se reemplaza una escritura del tipo:

```
#define RCC_APB2ENR (*(volatile uint32_t*)0x40021018)
RCC_APB2ENR |= (1 << 2);
```

por una escritura usando las estructuras del fabricante:

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
```

2. Estructura de trabajo

Se recomienda crear un directorio principal para el proyecto. Por ejemplo:

```
ej3_TP2/
|
|--- CMSIS/
|--- src/
|--- ld/
|--- Makefile
```

- **CMSIS/**: contiene los archivos de CMSIS y los archivos específicos del STM32F103.
- **src/**: contiene los archivos fuente de la aplicación, por ejemplo `main.c` y `syscalls.c`.
- **ld/**: contiene el linker script.
- **Makefile**: automatiza el proceso de compilación, linkeo y grabación.

3. Descarga de CMSIS Core

CMSIS es una librería promovida por ARM que define registros, funciones y estructuras estándar para trabajar con procesadores Cortex-M. Su principal ventaja es que permite migrar código entre distintos microcontroladores ARM con menor dependencia del fabricante.

La librería CMSIS puede descargarse desde:

<https://www.arm.com/technologies/cmsis>

Desde allí se accede al repositorio oficial. También puede descargarse directamente desde:

https://github.com/ARM-software/CMSIS_6

Una vez descargado el proyecto, dirigirse al directorio:

```
CMSIS_6/CMSIS/Core/Include/
```

y copiar al directorio CMSIS/ del proyecto los siguientes archivos:

```
cmsis_compiler.h  
cmsis_gcc.h  
cmsis_version.h  
core_cm3.h
```

Descripción de los archivos CMSIS Core

- `core_cm3.h`: define los registros y estructuras internas del núcleo Cortex-M3, como NVIC, SysTick y SCB.
- `cmsis_compiler.h`: contiene definiciones comunes para adaptar CMSIS a distintos compiladores.
- `cmsis_gcc.h`: contiene macros e instrucciones específicas para el compilador GCC, como `__NOP()`.
- `cmsis_version.h`: contiene información de versión de CMSIS.

Directorio adicional requerido en CMSIS

En versiones recientes de CMSIS, además de los archivos anteriores, también es necesario copiar el archivo:

```
CMSIS_6/CMSIS/Core/Include/m-profile/cmsis_gcc_m.h
```

en un nuevo directorio dentro de CMSIS, que llamaremos `m-profile`.

4. Descarga de CMSIS Device para STM32F1

CMSIS Core define los elementos generales del nucleo ARM Cortex-M, pero no incluye las definiciones particulares de cada microcontrolador. Para la placa BluePill se necesitan los archivos específicos de la familia STM32F1.

Estos archivos se descargan desde el repositorio de STMicroelectronics:

<https://github.com/STMicroelectronics/cmsis-device-f1>

Desde ese repositorio copiar al directorio CMSIS/ del proyecto:

```
stm32f1xx.h           //en cmsis-device-f1/Include/
stm32f103xb.h         //en cmsis-device-f1/Include/
system_stm32f1xx.h    //en cmsis-device-f1/Include/
system_stm32f1xx.c     //en cmsis-device-f1/Source/Templates/
startup_stm32f103xb.s  //en cmsis-device-f1/Source/Templates/gcc/
```

Descripcion de los archivos CMSIS Device

- **stm32f1xx.h**: archivo principal de la familia STM32F1. Segun el microcontrolador seleccionado, incluye el archivo correspondiente.
- **stm32f103xb.h**: define los perifericos y registros del STM32F103xB, como RCC, GPIOA, GPIOB, GPIOC, AFIO, EXTI, ADC1, etc.
- **system_stm32f1xx.h**: declara funciones y variables del sistema, como `SystemInit()`, `SystemCoreClock` y `SystemCoreClockUpdate()`.
- **system_stm32f1xx.c**: implementa la inicializacion del reloj del sistema. En nuestro caso se modifica para configurar el microcontrolador a 72 MHz.
- **startup_stm32f103xb.s**: archivo de arranque en assembler. Contiene la tabla de vectores, el `Reset_Handler` y la secuencia inicial de arranque. En los ejercicios bare metal previos se utilizaba un archivo propio, por ejemplo `crt.s`, que es reemplazado por el nuevo startup.

5. Linker script

El linker script indica al enlazador como ubicar las secciones del programa en la memoria física del microcontrolador. Copiar al directorio `ld/` el archivo:

```
STM32F103XB_FLASH.ld  //en cmsis-device-f1/Source/Templates/gcc/linker/
```

Este archivo reemplaza al `linker.ld` usado en los ejercicios bare metal anteriores y define, entre otras cosas:

- la dirección de inicio de la memoria FLASH,
- la dirección de inicio de la memoria RAM,
- la ubicación de la tabla de vectores,
- la ubicación de las secciones `.text`, `.rodata`, `.data` y `.bss`,
- los tamaños mínimos de heap y stack.

6. Archivo syscalls.c

El startup de ST llama a `__libc_init_array()`, funcion perteneciente a la librería `newlib`. Esta librería puede requerir algunas funciones mínimas del sistema, por ejemplo `_init()`, `_write()` o `_read()`.

Por este motivo se agrega el archivo:

```
CMSIS/syscalls.c
```

7. Makefile

El `Makefile` automatiza la compilacion, el linkeo y la generacion de los archivos finales. Debe compilar al menos los siguientes archivos:

```
src/main.c
CMSIS/syscalls.c
CMSIS/system_stm32f1xx.c
CMSIS/startup_stm32f103xb.s
```

Las opciones mas importantes son:

- `-mcpu=cortex-m3`: indica el nucleo del microcontrolador.
- `-mthumb`: genera codigo Thumb, usado por Cortex-M.
- `-DSTM32F103xB`: define el microcontrolador usado para que CMSIS incluya el archivo correcto.
- `-ICMSIS`: indica al compilador donde buscar los archivos de cabecera.
- `-T ld/STM32F103XB_FLASH.ld`: indica el linker script.
- `-nostartfiles`: evita usar los archivos de arranque por defecto de GCC, ya que se usa el startup de ST.
- `-Wl,-gc-sections`: elimina secciones no utilizadas.
- `-Wl,-Map=build/app.map`: genera el mapa de memoria.

8. Estructura final del proyecto

```
ej3_TP2/
|
|--- CMSIS/
|   |--- cmsis_compiler.h
|   |--- cmsis_gcc.h
|   |--- cmsis_version.h
|   |--- core_cm3.h
|   |--- stm32f1xx.h
|   |--- stm32f103xb.h
```

```

|--- system_stm32f1xx.h
|--- system_stm32f1xx.c
|--- startup_stm32f103xb.s
|--- syscalls.c
'--- m-profile/
    '--- cmsis_gcc_m.h
|--- src/
    '--- main.c
|--- ld/
    '--- STM32F103XB_FLASH.ld
'--- Makefile

```

9. Ayuda rápida – Equivalencias comunes Bare Metal vs CMSIS

La siguiente tabla resume algunas equivalencias frecuentes entre la programación bare metal utilizada en los ejercicios anteriores y la forma equivalente utilizando CMSIS.

Bare Metal manual	CMSIS
RCC_APB2ENR	RCC->APB2ENR
GPIOA_CRL	GPIOA->CRL
GPIOB_CRH	GPIOB->CRH
GPIOC_ODR	GPIOC->ODR
GPIOC_IDR	GPIOC->IDR
AFIO_EXTICR1	AFIO->EXTICR[0]
EXTI_IMR	EXTI->IMR
EXTI_RTSR	EXTI->RTSR
EXTI_FTSR	EXTI->FTSR
EXTI_PR	EXTI->PR
ADC1_CR1	ADC1->CR1
ADC1_CR2	ADC1->CR2
ADC1_SMPR1	ADC1->SMPR1
ADC1_SMPR2	ADC1->SMPR2
ADC1_SQR3	ADC1->SQR3
ADC1_DR	ADC1->DR
NVIC_ISERO	NVIC_EnableIRQ()

Observación:

Las definiciones de registros, estructuras y máscaras de bits utilizadas por CMSIS se encuentran principalmente en:

CMSIS/stm32f103xb.h

Allí pueden consultarse las equivalencias completas entre registros físicos, estructuras CMSIS y máscaras de bits del microcontrolador.

““latex

9.1. Equivalencias comunes de bits y máscaras

Además del acceso a registros mediante estructuras, CMSIS define nombres simbólicos para los bits más utilizados, evitando escribir desplazamientos manuales del tipo $(1 \ll n)$.

La siguiente tabla resume algunas equivalencias frecuentes utilizadas en los trabajos prácticos.

Bare Metal manual	CMSIS
$(1 \ll 0)$ AFIOEN	RCC_APB2ENR_AFIOEN
$(1 \ll 2)$ GPIOAEN	RCC_APB2ENR_IOPAEN
$(1 \ll 3)$ GPIOBEN	RCC_APB2ENR_IOPBEN
$(1 \ll 4)$ GPIOCEN	RCC_APB2ENR_IOPCEN
$(1 \ll 9)$ ADC1EN	RCC_APB2ENR_ADC1EN
$(1 \ll 0)$ ADON	ADC_CR2_ADON
$(1 \ll 1)$ CONT	ADC_CR2_CONT
$(1 \ll 22)$ SWSTART	ADC_CR2_SWSTART
$(1 \ll 23)$ TSVREFE	ADC_CR2_TSVREFE
$(1 \ll 1)$ EOC	ADC_SR_EOC
$(1 \ll 0)$ EXTI0	EXTI_IMR_MR0
$(1 \ll 0)$ PR0	EXTI_PR_PRO
$(1 \ll 13)$	GPIO_ODR_ODR13
$(1 \ll 13)$	GPIO_BSRR_BS13

Por ejemplo, en lugar de escribir:

```
RCC->APB2ENR |= (1 << 4);
```

con CMSIS puede escribirse:

```
RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
```

Ambas instrucciones realizan exactamente la misma operación, pero la segunda resulta más legible y reduce errores de interpretación. ““